合肥工業大學

专业课程

（计算机与信息学院）

大数据处理技术

课程报告

专 业 班 级 计算机科学与技术20-3班

学生姓名及学号 雷俊杰 2020215470

课 程 教 学 班 号 0509880X--001

实验序号及名称 课程报告——实验指导

任 课 教 师 吴共庆

实 验 指 导 教 师 吴共庆

2021～2022学年第 二 学期

说　明

实验报告是关于实验教学内容、过程及效果的记录和总结，因此，应注意以下事项和要求：

1．每个实验单元在50页的篇幅内完成一份报告。“实验单元”指按照实验指导书规定的实验内容。若篇幅不够，可另附纸。

2．实验报告要求：**书写工整规范，语言表达清楚，数据和程序真实。理论联系实际，认真分析实验中出现的问题与现象，总结经验。**

3．参加实验的每位同学应独立完成实验报告的撰写，其中程序或相关的设计图纸也可以采用打印等方式粘贴到报告中。严禁抄袭或拷贝，否则，一经查实，按作弊论取，并取消理论课考试资格。

4．实验报告作为评定实验成绩的依据。

实验序号及名称：实验_课程报告_ _撰写大数据实验指导书_

实验时间： 2022年_6_月_25_日

选择题目：

“自学一项大数据相关技术，按照本课程实验指导格式撰写一份实验教程。”。

附有目录如下：

目录

1 Docker简介 5
1.1 什么是Docker 5
1.2 为什么要使用Docker 5
1.3 Docker组成 6
1.4 Docker容器技术与虚拟机比较 8
2 Docker安装 9
2.1 系统要求 9
2.2 安装依赖设置yum仓库 9
2.3 开始安装Docker 9
2.4 启动Docker 10
2.5 验证是否安装成功 10
3 Dokcer镜像的使用 11
3.1 获取镜像 11
3.2 查看镜像 12
3.3 搜寻镜像 14
3.4 删除和清理镜像 15
3.5 创建镜像 17
3.6 存出和存入镜像 18
3.7 上传镜像 18
4 Docker容器的操作 19
4.1 创建容器 19
4.2 停止容器 21
4.3 进入容器 22
4.4 删除容器 22
4.5 导入和导出容器 23
5 Docker搭建大数据平台 25
5.1 Hadoop 25
5.2 Spark 28
5.3 Storm 30
6 写在最后 32
6.1 心得体会 32

6.2 参考材料 32

1 Docker简介

1.1 什么是Docker

docker是一个用Go语言实现的开源项目，可以让我们方便的创建和使用容器，docker将程序以及程序所有的依赖都打包到docker container，这样你的程序可以在任何环境都会有一致的表现，这里程序运行的依赖也就是容器就好比集装箱，容器所处的操作系统环境就好比货船或港口，程序的表现只和集装箱有关系(容器)，和集装箱放在哪个货船或者哪个港口(操作系统)没有关系。

因此我们可以看到docker可以屏蔽环境差异，也就是说，只要你的程序打包到了docker中，那么无论运行在什么环境下程序的行为都是一致的，程序员再也无法施展表演才华了，不会再有“在我的环境上可以运行”，真正实现“build once，run everywhere”。

此外docker的另一个好处就是快速部署，这是当前互联网公司最常见的一个应用场景，一个原因在于容器启动速度非常快，另一个原因在于只要确保一个容器中的程序正确运行，那么你就能确信无论在生产环境部署多少都能正确运行。

1.2 为什么要使用Docker

先说Docker干啥用的。因为现在物理服务器是很强大的，我们如果在一台物理服务器上只跑一个服务就浪费了，而同时跑很多服务他们又互相影响；比如说一个服务出了内存泄漏把整个服务器的内存都占满了，其他服务都跟着倒毒。所以要把每个服务都隔离起来，让它们只使用自己那部分有限的cpu,内存和磁盘，以及自己依赖的软件包。这个早先是用虚拟机来实现隔离的，但是每个虚拟机都要装自己的操作系统核心，这是对资源有点浪费。于是就有了Docker，一个机器上可以装十几个到几十个docker,他们共享操作系统核心，占用资源少，启动速度快。但又能提供了资源(cpu,内存，磁盘等)的一定程度的隔离。

1.3 Docker组成

Docker由3大核心组成：镜像、容器、仓库，大部分的操作都是围绕着这三大核心概念

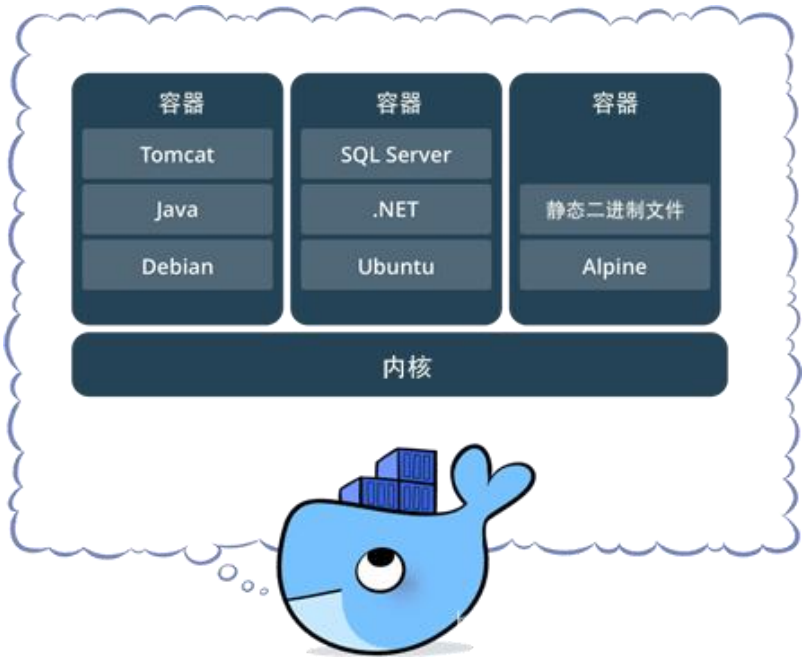


图 1 Docker直观图示

如上图所示鲸鱼是操作系统，要交付的应用程序是各种货物，要将各种形状和尺寸不同的货物放到大鲸鱼上，得考虑每件货物怎么安放（应用程序配套的环境），还得考虑货物和货物之间能否重叠起来（应用程序依赖的环境是否会冲突）。

现在使用了集装箱（容器）把每件货物都放到集装箱里，这样大鲸鱼可以用同样地方式安放、堆叠集装了,省事省力。即:打包放到鲸鱼上,鲸鱼放到服务器上。也就是“build——ship——run”，这样在自己的电脑上怎么运行，在服务器上也会怎么运行。

用docker运行一个程序的过程：去仓库把镜像拉到本地，然后用一条命令把镜像运行起来，变成容器。

- **Docker镜像**

Docker 镜像类似于虚拟机镜像，可以将它理解为一个只读的模板。

例如一个镜像可以包含一个基本的操作系统环境，里面仅安装了 Apache 应用程序。可以把它称为一个 Apache 镜像，镜像是创建 Docker容器的基础。

通过版本管理和增量的文件系统，Docker提供了一套十分简单的机制来创建和更新现有镜像，用户甚至可以从网上下载一个已经做好的应用镜像并直接使用。

- **Docker容器**

Docker 容器类似于一个轻量级的沙箱，Docker 利用容器来运行和隔离应用。

容器是从镜像创建的应用运行实例。它可以启动、开始、停止、删除，而这些容器都是彼此相互隔离，互不可见的。

- **Docker仓库**

Docker仓库类似于代码仓库，是Docker集中存放镜像文件的场所。

仓库注册服务器是存放仓库的地方，骑上往往存放着多个仓库。每个仓库集中存放某一类镜像，往往包括多个镜像文件，通过不同的标签来进行区分。

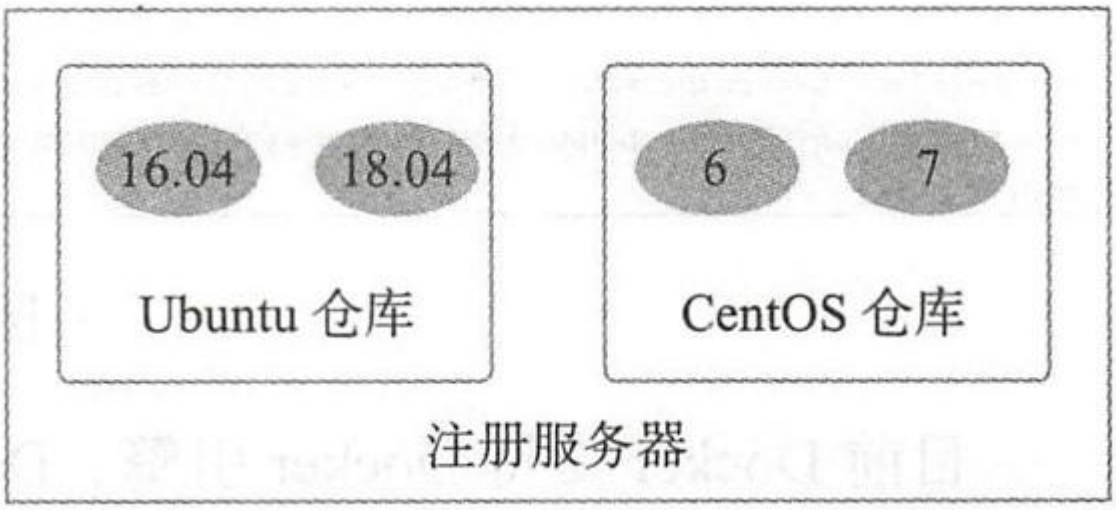


图 2 注册服务器与仓库

同时，根据所存储的镜像公开分享与否，Docker 仓库可以分为公开仓库和私有仓库两种形式。

1.4 Docker容器技术与虚拟机比较

作为一种轻量级的虚拟化方式,Docker 在运行应用上跟传统的虚拟机方式相比具有如下显著优势:

• Docker容器很快，启动和停止可以在秒级实现，这相比传统的虚拟机方式（数分钟）要快得多;

• Docker容器对系统资源需求很少，一台主机上可以同时运行数千个Docker容器;

• Docker通过类似Git设计理念的操作来方便用户获取、分发和更新应用镜像，存储复用，增量更新;

•Docker通过 Dockerfile支持灵活的自动化创建和部署机制,以提高工作效率,并标准化流程。

整体对比如下表所示

表 1 Docker 容器与传统虚拟机技术比较

特性	Docker容器	传统虚拟机
启动速度	秒级	分钟级
性能	接近原生	较弱
内存代价	很小	较多
硬盘使用	一般为MB	一般为GB
运行速度	单机支持上千个容器	一般只有几十个
隔离性	安全隔离	完全隔离
迁移性	优秀	一般

2 Docker安装

2.1 系统要求

Docker 要求 CentOS 系统的内核版本高于 3.10,查看本页面的前提条件来验证你的CentOS 版本是否支持 Docker 。

通过 uname -r 命令查看你当前的内核版本

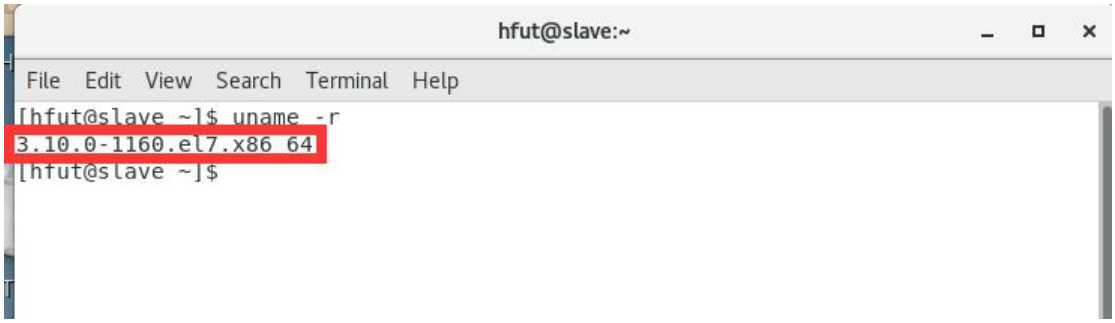


图 3 查看内核版本

一般说来系统内核越新越好

必须启用存储库。默认情况下，此存储库处于启用状态，但如果已禁用它，则需要重新启用它。

```
centos-extras
```

2.2 安装依赖设置yum仓库

安装依赖:

```
yum install -y yum-utils \
device-mapper-persistent-data \
lvm2
```

设置仓库:

```
yum-config-manager \
--add-repo \
https://download.docker.com/linux/centos/docker-ce.repo
```

2.3 开始安装Docker

安装最新版本的Docker CE

```
yum install docker-ce docker-ce-cli containerd.io
```

执行这个命令要耐心等待，中间要输入两次y，当然你也可以在yum命令后面直接跟-y跳过询问

2.4 启动Docker

启动命令如下：

```
systemctl start docker
```

加入开机启动（可做可不做）

```
systemctl enable docker
```

2.5 验证是否安装成功

1. 查看版本

```
docker version
```

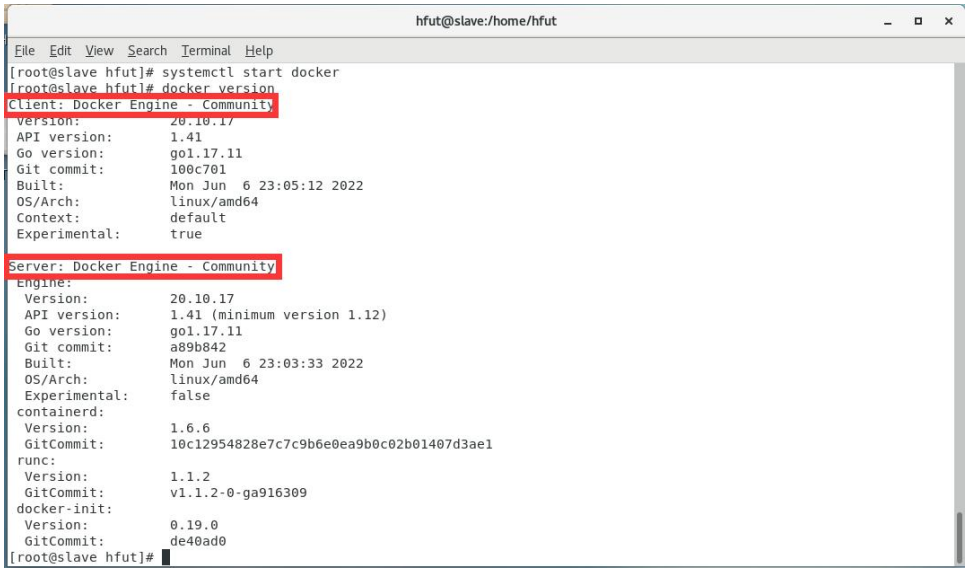


图 4 查看版本

2. 运行hello-world校验Docker安装是否成功

```
docker run hello-world
```

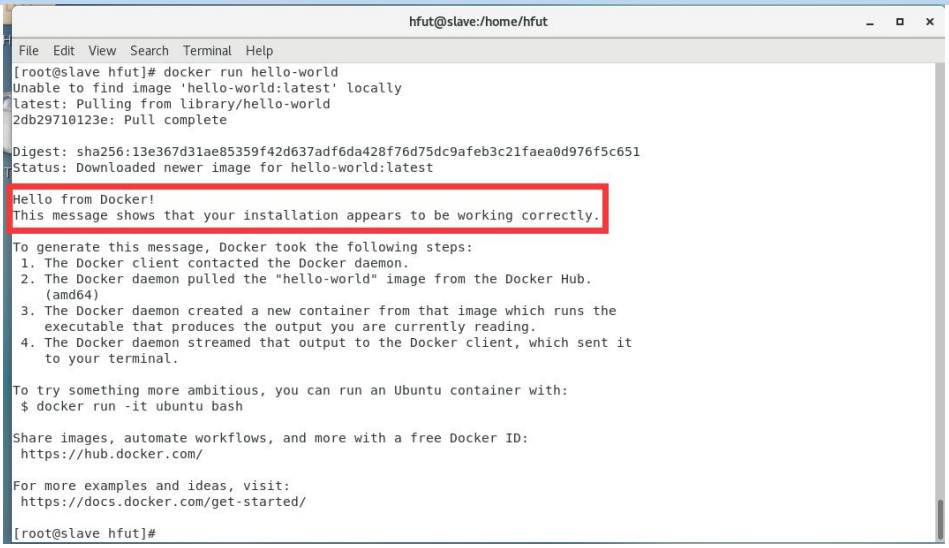


图 5 运行hello-world

3 Dokcer镜像的使用

Docker运行容器前需要本地存在对应的镜像，如果镜像不存在，Docker会尝试先从默认镜像仓库下载（默认使用 Docker Hub 公共注册服务器中的仓库），用户也可以通过配置，使用自定义的镜像仓库。

3.1 获取镜像

镜像是运行容器的前提，官方的Docker Hub网站已经提供了数十万个镜像供用户开放下载。

此小节介绍 Docker镜像的pull子命令。

可以使用 docker [image] pull 命令直接从 Docker Hub 镜像源来下载镜像。该命令的格式为 docker [image] pull NAME [:TAG]。

- NAME：镜像仓库名称（用来区分镜像）
- TAG：镜像的标签（用来表示版本）

对于 Docker 镜像来说，如果不显式指定 TAG，则默认会选择 latest 标签，这会下载仓库中最新版本的镜像。

下例通过 latest 标签下载最新版本的 centos 操作系统的镜像：

```
docker pull centos
```

```
[root@slave hfut]# docker pull centos
Using default tag: latest
latest: Pulling from library/centos
a1d0c7532777: Pull complete
Digest: sha256:a27fd8080b517143cbbbab9dfb7c8571c40d67d534bbdee55bd6c473f432b177
Status: Downloaded newer image for centos:latest
docker.io/library/centos:latest
[root@slave hfut]#
```

图 6 获取centos最新版本

下载过程中可以看出，镜像文件一般由若干层(layer)组成，6c953ac5d795这样的串是层的唯一id（实际上完整的 id 包括 256 比特，64 个十六进制字符组成）。使用 docker pull 命令下载中会获取并输出镜像的各层信息。当不同的镜像包括相同的层时，本地仅存储层的一份内容，减小存储空间。

在不同的镜像仓库服务器的情况下，可能会出现镜像重名的情况。严格地讲，镜像的仓库名称中还应该添加仓库地址(即registry，注册服务器)作为前缀，只是默认使用的是官方 Docker Hub 服务，该前缀可以忽略。如果从非官方的仓库下载，则需要在仓库名称前指定完整的仓库地址。

子命令支持的选项主要包括：

- -a, --all-tags=true|false: 是否获取仓库中的所有镜像，默认为否
- --disable-content-trust: 取消镜像的内容校验，默认为真。

有时需要使用镜像代理服务来加速 Docker 镜像获取过程，可以在 Docker 服务启动配置中增加 --registry-mirror=proxy_URL 来指定镜像代理服务地址（如 https://registry.docker.com）

3.2 查看镜像

1. 使用 images 命令列出镜像

使用如下命令可以列出本地主机上已有镜像的基本信息。

```
docker images
```

或者：

```
docker images ls
```

下方图片列出了上节下载的镜像信息：

```
[root@slave hfut]# docker images
REPOSITORY    TAG       IMAGE ID       CREATED         SIZE
hello-world   latest    feb5d9fea6a5   9 months ago    13.3kB
centos        latest    5d0da3dc9764   9 months ago    231MB
[root@slave hfut]#
```

图 7 镜像信息

在列出的信息中，可以看到如下几个字段信息：

- REPOSTTORY: 来自于哪个仓库
- TAG: 镜像的标签信息
- IMAGE ID: 镜像的 ID（唯一标识镜像）
- CREATED: 创建时间
- SIZE: 镜像大小

其中镜像的 ID 信息十分重要，它唯一标识了镜像。在使用镜像 ID 的时候，一般可以使用该 ID 的前若干个字符组成的可区分串来替代完整的 ID。如果两个镜像的 ID 相同，说明它们实际上指向了同一个镜像，只是具有不同标签名称而已。

TAG 信息用于标记来自同一个仓库的不同镜像。例如 centos 仓库中有多个镜像，就可以通过TAG 信息来区分发行版本。

镜像大小信息只是表示了该镜像的逻辑体积大小，实际上由于相同的镜像层本地只会存储一份，物理上占用的存储空间会小于各镜像逻辑体积之和。

更多子命令可通过下方命令查看：

```
man docker-images
```

2. 添加镜像标签

为了方便在后续工作中使用特定镜像，还可以使用 docker tag 命令来为本地镜像任意添加新的标签。例如，添加一个新的mycentos:latest 镜像标签：

```
docker tag centos:latest mycentos:latest
```

再次使用 docker images 列出本地主机上镜像信息，可以看到多了一个 mycentos:latest 标签的镜像：

```
[root@slave hfut]# docker tag centos:latest mycentos:latest
[root@slave hfut]# docker images
REPOSITORY    TAG       IMAGE ID       CREATED         SIZE
hello-world   latest    feb5d9fea6a5   9 months ago    13.3kB
centos        latest    5d0da3dc9764   9 months ago    231MB
mycentos      latest    5d0da3dc9764   9 months ago    231MB
[root@slave hfut]#
```

图 8 为centos:latest 添加标签

docker tag 命令添加的标签实际上起到了类似链接的作用。

3. 查看详细信息

使用 docker inspect [image] 命令可以获取该镜像的详细信息，包括制作者、适应架构、各层的数字摘要等：

```
docker inspect centos:latest
```

```
[root@slave hfut]# docker inspect centos:latest
[
    {
        "Id": "sha256:5d0da3dc976460b72c77d94c8a1ad043720b0416bfc16c52c45d4847e53fadb6",
        "RepoTags": [
            "centos:latest",
            "mycentos:latest"
        ],
        "RepoDigests": [
            "centos@sha256:a27fd8080b517143cbbbab9dfb7c8571c40d67d534bbdee55bd6c473f432b177"
        ],
        "Parent": "",
        "Comment": "",
        "Created": "2021-09-15T18:20:05.184694267Z",
        "Container": "9bf8a9e2ddff4c0d76a587c40239679f29c863a967f23abf7a5babb6c2121bf1",
        "ContainerConfig": {
            "Hostname": "9bf8a9e2ddff",
            "Domainname": "",
            "User": "",
            "AttachStdin": false,
            "AttachStdout": false,
            "AttachStderr": false,
            "Tty": false,
            "OpenStdin": false,
            "StdinOnce": false,
            "Env": [
                "PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"
            ],
            "Cmd": [
                "/bin/sh",
                "-c",
                "#(nop) ",
                "CMD [\"/bin/bash\"]"
            ],
            "Image": "sha256:f5b050f177fd426be8fe998a8ecf3fb1858d7e26dff4080b29a327d1bd5ba422",
            "Volumes": null,
            "WorkingDir": "",
            "Entrypoint": null,
hfut@slave:/home/hfut
```

图 9 获取 centos:latest 详细信息

4. 查看镜像历史

使用 history 子命令可列出各层的创建信息。下例将查看 centos:latest 镜像的创建过程：

```
docker history centos:latest
```

```
[root@slave hfut]# docker history centos:latest
IMAGE          CREATED        CREATED BY                                      SIZE      COMMENT
5d0da3dc9764   9 months ago   /bin/sh -c #(nop)  CMD ["/bin/bash"]            0B
<missing>      9 months ago   /bin/sh -c #(nop)  LABEL org.label-schema.sc…   0B
<missing>      9 months ago   /bin/sh -c #(nop) ADD file:805cb5e15fb6e0bb0…   231MB
[root@slave hfut]#
```

图 10 获取centos:latest 镜像创建过程

3.3 搜寻镜像

使用 docker search 命令可以搜索 Docker Hub 官方仓库中的镜像。语法为 docker search [option] keyword。支持的命令选项主要包括：

- -f, -filter filter: 过滤输出内容；
- -format string: 格式化输出内容；
- -limit int: 限制输出结果个数，默认为 25 个；
- -no-trunc: 不截断输出结果。

如下命令将搜索官方提供的带 nginx 关键字的镜像：

```
docker search --filter=is-official=true nginx
```

```
[root@slave hfut]# docker search --filter=is-official=true nginx
NAME      DESCRIPTION                 STARS     OFFICIAL   AUTOMATED
nginx     Official build of Nginx.    17072     [OK]
[root@slave hfut]#
```

图 11 获取 nginx 官方镜像

如下命令将搜索所有收藏数超过 4 的关键词包括 tensorflow 的镜像：

```
docker search --filter=stars=4 tensorflow
```

```
[root@slave hfut]# docker search --filter=stars=4 tensorflow
NAME                                DESCRIPTION                                     STARS     OFFICIAL   AUTOMATED
tensorflow/tensorflow               Official Docker images for the machine learn…   2031
tensorflow/serving                  Official images for TensorFlow Serving (http…   123
bitnami/tensorflow-serving          Bitnami Docker Image for TensorFlow Serving     27                   [OK]
armswdev/tensorflow-arm-neoverse-n1 Please use "armswdev/tensorflow-arm-neoverse"   10
ibmcom/tensorflow-ppc64le           Community supported ppc64le docker images fo…   6
intel/intel-optimized-tensorflow    Containers with TensorFlow* optimized with o…   6
armswdev/tensorflow-arm-neoverse    TensorFlow builds for AArch64 CPUs              4
[root@slave hfut]#
```

图 12 获取收藏数超过4的tensorflow镜像

3.4 删除和清理镜像

1. 使用标签删除镜像

使用 docker rmi 或 docker image rm 命令可以删除镜像，命令格式为 docker rmi IMAGE [IMAGE...]，其中 IMAGE 可以为标签或 ID。

支持选项包括：

- -f，-force：强制删除镜像，即使有容器依赖它；
- -no- prune：不要清理未带标签的父镜像。

如果要删除 mycentos:latest 镜像，可使用如下命令：

```
docker rmi mycentos:latest
```

```
[root@slave hfut]# docker rmi mycentos:latest
Untagged: mycentos:latest
[root@slave hfut]# docker images
REPOSITORY    TAG       IMAGE ID       CREATED         SIZE
hello-world   latest    feb5d9fea6a5   9 months ago    13.3kB
centos        latest    5d0da3dc9764   9 months ago    231MB
[root@slave hfut]#
```

图 13 删除 mycentos：latest 镜像

当同一个镜像拥有多个标签的时候，docker rmi 命令只是删除了该镜像多个标签中的指定标签而已，并不影响镜像文件。但当镜像只剩下一个标签时，时再使用 docker rmi 命令会彻底删除镜像。

下例通过执行 docker rmi 删除只有一个标签的镜像，可以看出会删除这个镜像文件的所有文件层。

```
docker rmi busybox:latest
```

```
[root@slave hfut]# docker pull busybox
Using default tag: latest
latest: Pulling from library/busybox
19d511225f94: Pull complete
Digest: sha256:3614ca5eacf0a3a1bcc361c939202a974b4902b9334ff36eb29ffe9011aaad83
Status: Downloaded newer image for busybox:latest
docker.io/library/busybox:latest
[root@slave hfut]# docker images
REPOSITORY    TAG       IMAGE ID       CREATED         SIZE
busybox       latest    62aedd01bd85   4 weeks ago     1.24MB
hello-world   latest    feb5d9fea6a5   9 months ago    13.3kB
centos        latest    5d0da3dc9764   9 months ago    231MB
[root@slave hfut]# docker rmi busybox:latest
Untagged: busybox:latest
Untagged: busybox@sha256:3614ca5eacf0a3a1bcc361c939202a974b4902b9334ff36eb29ffe9011aaad83
Deleted: sha256:62aedd01bd8520c43d06b09f7a0f67ba9720bdc04631a8242c65ea995f3ecac8
Deleted: sha256:7ad00cd55506625f2afad262de6002c8cef20d214b353e51d1025e40e8646e18
[root@slave hfut]#
```

图 14 下载并删除 busybox:latest 镜像

2. 使用镜像 ID 来删除镜像

当使用 docker rmi 命令，并且后面跟上镜像的 ID 时，会先尝试删除所有指向该镜像的标签，然后删除该镜像文件本身。

注意，当该镜像创建的容器存在时，镜像文件默认是无法被删除的，例如：先

利用 centos:latest 镜像创建一个简单的容器来输出一段话：

```
docker run centos:latest echo 'hello! I want to go home ！'
```

```
[root@slave hfut]# docker run centos:latest echo 'hello! I want to go home ！'
hello! I want to go home ！
[root@slave hfut]#
```

图 15 创建容器

使用如下命令可以看到本机上存在的所有容器：

```
docker ps -a
```

```
[root@slave hfut]# docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED              STATUS                          PORTS     NAMES
ffc02ff38c83   centos:latest  "echo 'hello! I want…"   About a minute ago   Exited (0) About a minute ago             amazing_aryabhata
448feb4421db   centos:latest  "echo 'hello! I am h…"   About a minute ago   Exited (0) About a minute ago             amazing_carver
8dd3224ae424   hello-world    "/hello"                 27 minutes ago       Exited (0) 27 minutes ago                 amazing_montalcini
[root@slave hfut]#
```

图 16 查看本机容器

可以看到，后台存在一个退出状态的容器，是刚基于 centos:latest镜像创建的。试图删除该镜像，Docker 会提示有容器正在运行，无法删除：

```
docker rmi centos:latest
```

```
[root@slave hfut]# docker rmi centos:latest
Error response from daemon: conflict: unable to remove repository reference "centos:latest" (must force) - container 448feb4421db is using its referenced image 5d0da3dc9764
[root@slave hfut]#
```

图 17 当容器存在时无法删除

如果要想强行删除镜像，可以使用 -f 参数：

```
docker rmi -f centos:latest
```

正常的删除镜像的做法是，先删除依赖该镜像的所有容器，再来删除镜像。

3. 清理镜像

使用 Docker 一段时间后，系统中可能会遗留一些临时的镜像文件，以及一些没有被使用的镜像，可以通过如下命令进行清理：

```
docker image prune
```

```
[root@slave hfut]# docker image prune
WARNING! This will remove all dangling images.
Are you sure you want to continue? [y/N] y
Total reclaimed space: 0B
[root@slave hfut]#
```

图 18 清理镜像

支持选项包括：

- -a, -all ： 删除所有无用镜像，不光是临时镜像；
- filter filter ： 只清理符合给定过滤器的镜像；

- -f, -force ：强制删除镜像，而不进行提示确认。

3.5 创建镜像

创建镜像主要有三种方法：基于已有镜像的容器创建、基于本地模板导入、基于 Dockerfile 创建。

1. 基于已有镜像的容器创建

该方法主要是使用 docker [container] comit 命令，命令格式如下：

```
docker [container] commit [OPTIONS] CONTAINER [REPOSITORY[:TAG]]
```

主要选项包括：

- -a, -author=”” ：作者信息；
- -c, -change-[]：提交的时候执行 Dockerfile 指令
- -m, -message=”” ：提交消息；
- -p, -pause-true：提交时暂停容器运行。

演示如下，首先启动一个镜像，并在其中进行修改操作。如创建一个 test 文件后退出，代码如下：

```
docker run -it centos:latest /bin/bash
/# touch test
/# exit
```

此时容器 ID 为 441984f4416d。

此时该容器与原 centos:latest 镜像相比，已经发生了改变，可以使用 docker commit 命令来提交为一个新的镜像。提交时可以使用 ID 或名称来指定容器：

```
docker commit -a "Cage" -m "a new file" 441984f4416d test:0.1
```

如果创建顺利，则会返回新创建镜像的 ID 信息，此时查看本地镜像列表，会发现新创建的镜像已经存在。

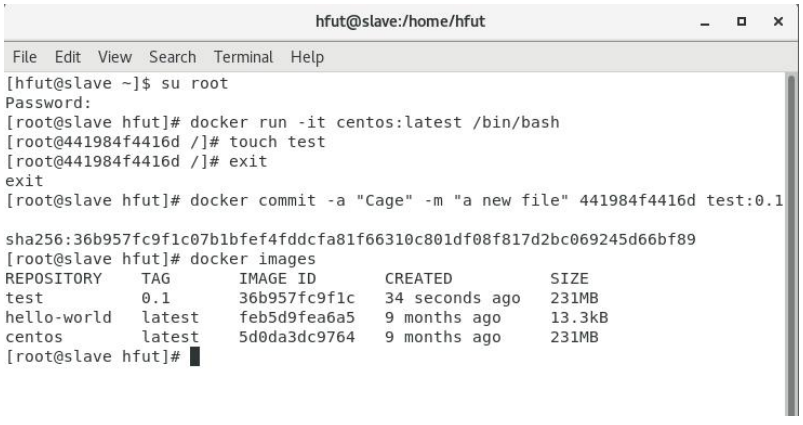


图 19 创建新镜像结果

2.基于本地模板导入

用户也可直接从一个操作系统模板文件导入一个镜像，主要使用 docker [container] import 命令。比如，先下载了一个 centos-7-x86_64-minimal.tar.gz 的镜像，之后使用以下命令导入：

```
cat centos-7-x86_64-minimal.tar.gz | docker import - centos:v7.mini
```

3.基于 Dockerfile 创建

基于 Dockerfile 创建是最常见的方式。Dockerfile 是一个文本文件，利用给定的指令描述基于某个父镜像创建新镜像的过程。

3.6 存出和存入镜像

1. 存出镜像

如果要导出镜像到本地文件，可以使用 docker [image] save 命令。下例将导出本地的 centos:latest 镜像为文件 centos 7.0.tar：

```
docker save -o ~/centos 7.0.tar centos:latest
```

2. 存入镜像

可以使用 docker [image] load 将导出的 tar 文件再导入到本地镜像库。下例将从文件 centos 7.0.tar 导入镜像到本地镜像列表：

```
docker load -i centos 7.0.tar
```

或者：

```
docker load < centos 7.0.tar
```

这将导入镜像及其相关的元数据信息（包括标签等）。

3.7 上传镜像

如果要将镜像上传至仓库，可以使用 docker [image] push 命令，默认上传至 Docker Hub 仓库。下例将本地的 centos:latest 上传至仓库（**需登陆**）：

```
docker tag centos:latest Cage/centos:latest
docker push Cage/centos:latest
```

4 Docker容器的操作

容器是镜像的一个运行实例。所不同的是，镜像是静态的只读文件，而容器带有运行时需要的可写文件层，同时，容器中的应用进程处于运行状态。

如果认为虚拟机是模拟运行的整套操作系统（包括内核、应用运行态环境和其他系统环境）和跑在上面的应用。那么 Docker 容器就是独立运行的个（或组）应用，以及它们必需的运行环境。

4.1 创建容器

1. 新建容器

可以使用 docker [container] create 命令新建一个容器，例如：

```
$ docker create -it centos:latest
$ docker ps -a
```

```
[root@slave hfut]# docker create -it centos:latest
8b54742afce8e6c8229dfc9431111f73cd79d31b8182309caea9a49ca83c7684
[root@slave hfut]# docker ps -a
CONTAINER ID   IMAGE           COMMAND                  CREATED              STATUS
                      PORTS     NAMES
8b54742afce8   centos:latest   "/bin/bash"              24 seconds ago       Create
d                                sharp_ellis
441984f4416d   centos:latest   "/bin/bash"              48 minutes ago       Exited
 (0) 47 minutes ago              happy_greider
ce7a752a1115   centos:latest   "/bin/bash"              49 minutes ago       Exited
 (0) 7 minutes ago               flamboyant_easley
ffc02ff38c83   centos:latest   "echo 'hello! I want…"   55 minutes ago       Exited
 (0) 55 minutes ago              amazing_aryabhata
448feb4421db   centos:latest   "echo 'hello! I am h…"   56 minutes ago       Exited
 (0) 56 minutes ago              amazing_carver
8dd3224ae424   hello-world     "/hello"                 About an hour ago    Exited
 (0) About an hour ago           amazing_montalcini
[root@slave hfut]#
```

图 20 新建容器

使用 docker [container) create 命令新建的容器处于停止状态，可以使用 docker [container] start 命令来启动它。

2. 启动容器

使用 docker [container] start 命令来启动一个已经创建的容器，例如启动刚刚创建的 mysql 容器（通过容器 ID 启动）：

```
docker start 8b54742afce8
```

```
[root@slave hfut]# docker start 8b54742afce8
8b54742afce8
[root@slave hfut]# docker ps
CONTAINER ID   IMAGE          COMMAND       CREATED         STATUS              PORTS     NAMES
8b54742afce8   centos:latest  "/bin/bash"   3 minutes ago   Up About a minute             sharp_ellis
[root@slave hfut]#
```

图 21 启动容器

3. 新建并启动容器

使用 docker [container] run 命令新建并启动容器，等价于先执行 docker [container] create，再执行 docker [container] start 命令。

下例将输入“Hello World!”，之后容器会自动终止：

```
docker run centos:latest /bin/echo 'Hello World'
```

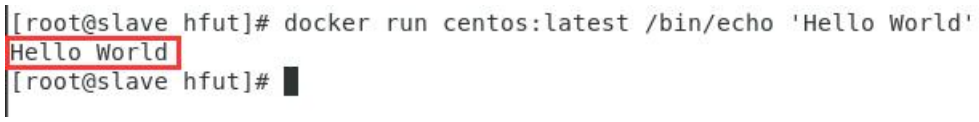


图 22 新建并启动容器

当利用 docker [container] run 创建并启动容器时，Docker 在后台运行的标准操作包括：

- 检查本地是否存在指定的镜像，不存在就从公有仓库下载；
- 利用镜像创建一个容器，并启动该容器；
- 分配一个文件系统给容器，并在只读的镜像层外面挂载层可读写层；
- 从宿主主机配置的网桥接口中桥接一个虚拟接口到容器中去；
- 从网桥的地址池配置一个 IP 地址给容器；
- 执行用户指定的应用程序；
- 执行完毕后容器被自动终止。

下例将启动一个 bash 终端，允许用户进行交互：

```
docker run -it centos:latest /bin/bash
```

其中，-t 选项让 Docker 分配一个伪终端并绑定到容器的标准输入上，-i 则让容器的标准输入保持打开。在交互模式下，用户可以通过所创建的终端来输入命令。

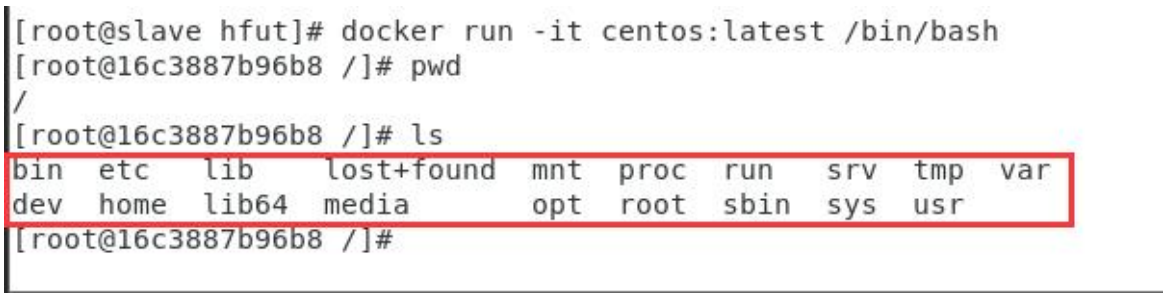


图 23 通过终端输入命令

使用完毕后通过输入 exit 命令来退出容器。

4. 守护态运行

更多的情况下需要让 Docer 容器在后台以守护态的形式运行，可以通过 -d 参数运行，例子如下：

```
docker run -d centos:latest /bin/echo 'Hello World'
```

```
[root@slave hfut]# docker run -d centos:latest /bin/echo 'Hello World'
3c7e9c1a6889e4c25ece2fb075479f970791471b9e6c963a2b14d40f4a5cc52b
[root@slave hfut]# docker ps
CONTAINER ID   IMAGE           COMMAND       CREATED          STATUS          PORTS     NAMES
8b54742afce8   centos:latest   "/bin/bash"   14 minutes ago   Up 12 minutes             sharp_ellis
[root@slave hfut]#
```

图 24 守护态运行容器

查看容器输出

要获取容器的输出信息，可以通过 docker [container] logs 命令，例子如下：

```
docker logs 8b54742afce8
```

4.2 停止容器

1. 暂停容器

使用 docker [container] pause CONTAINER [CONTAINER...] 命令来暂停一个运行中的容器，例子如下：

```
docker pause 8b54742afce8
docker ps
```

```
[root@slave hfut]# docker pause 8b54742afce8
8b54742afce8
[root@slave hfut]# docker ps
CONTAINER ID   IMAGE           COMMAND       CREATED          STATUS                     PORTS     NAMES
8b54742afce8   centos:latest   "/bin/bash"   21 minutes ago   Up 19 minutes (Paused)             sharp_ellis
[root@slave hfut]#
```

图 25 暂停容器

处于 paused 状态的容器可以使用 docker [container] unpause CONTAINER [CONTAINER...] 命令来恢复到运行状态。

2.终止容器

可以使用 docker [container] stop 来终止一个运行中的容器。该命令的格式为：

```
docker [container] stop [-t]--time[=10] [CONTAINER...]
```

该命令会首先向容器发送 SIGTERM 信号，等待一段超时时间后（默认为 10 秒），再发送 SIGKILL 信号来终止容器：

```
docker stop test
```

此时，执行 docker container prune 命令，会自动清除掉所有处于停止状态的容器。处于终止状态的容器，可以通过 docker [container] start 命令来重新启动。

4.3 进入容器

在使用 -d 参数时，容器启动后会进入后台，用户无法看到容器中的信息，也无法进行操作。这时如果需要进入容器进行操作，可以使用官方的 exec 命令。例子如下，进入到刚创建的容器中，并启动一个 bash：

```
docker run -itd centos
docker exec -it eb2504e5715f /bin/bash
```

```
[root@slave hfut]# docker run -itd centos
eb2504e5715fee95a329f386f2680c690ab7447e3531c519e869be91333d42b1
[root@slave hfut]# docker exec -it eb2504e5715f /bin/bash
[root@eb2504e5715f /]#
```

图 26 exec 命令

可以看到会打开一个新的 bash 终端，在不影响容器内其他应用的前提下，用户可以与容器进行交互。

4.4 删除容器

可以使用 docker [container] rm 命令来删除处于终止或退出状态的容器，命令格式为：

```
docker [container] rm [-f|-- force] [-l |--link][-v|--volumes] CONTAINER [CONTAINER...]
```

- -f，--force=false：是否强行终止并删除一个运行中的容器；
- -l，--link=false：删除容器的连接，但保留容器；
- -v，--volumes=false：删除容器挂载的数据卷。

查看处于终止状态的容器，并将其删除：

```
docker ps -a
docker rm 3c7e9c1a6889
```

```
[root@slave hfut]# docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED              STATUS                          PORTS     NAMES
eb2504e5715f   centos         "/bin/bash"              About a minute ago   Up About a minute                         lucid_shockley
3c7e9c1a6889   centos:latest  "/bin/echo 'Hello Wo…"   10 minutes ago       Exited (0) 10 minutes ago                 confident_euler
16c3887b96b8   centos:latest  "/bin/bash"              14 minutes ago       Exited (127) 10 minutes ago               silly_brown
7e82e2a75b5f   centos:latest  "/bin/echo 'Hello Wo…"   19 minutes ago       Exited (0) 19 minutes ago                 thirsty_carson
4e7bdcec216a   centos         "/bin/echo 'Hello Wo…"   19 minutes ago       Exited (0) 19 minutes ago                 objective_curran
8b54742afce8   centos:latest  "/bin/bash"              25 minutes ago       Up 23 minutes (Paused)                    sharp_ellis
441984f4416d   centos:latest  "/bin/bash"              About an hour ago    Exited (0) About an hour ago              happy_greider
ce7a752a1115   centos:latest  "/bin/bash"              About an hour ago    Exited (0) 32 minutes ago                 flamboyant_easley
ffc02ff38c83   centos:latest  "echo 'hello! I want…"   About an hour ago    Exited (0) About an hour ago              amazing_aryabhata
448feb4421db   centos:latest  "echo 'hello! I am h…"   About an hour ago    Exited (0) About an hour ago              amazing_carver
8dd3224ae424   hello-world    "/hello"                 2 hours ago          Exited (0) 2 hours ago                    amazing_montalcini
[root@slave hfut]# docker rm 3c7e9c1a6889
3c7e9c1a6889
[root@slave hfut]# docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED              STATUS                          PORTS     NAMES
eb2504e5715f   centos         "/bin/bash"              3 minutes ago        Up 3 minutes                              lucid_shockley
16c3887b96b8   centos:latest  "/bin/bash"              15 minutes ago       Exited (127) 11 minutes ago               silly_brown
7e82e2a75b5f   centos:latest  "/bin/echo 'Hello Wo…"   20 minutes ago       Exited (0) 20 minutes ago                 thirsty_carson
4e7bdcec216a   centos         "/bin/echo 'Hello Wo…"   20 minutes ago       Exited (0) 20 minutes ago                 objective_curran
8b54742afce8   centos:latest  "/bin/bash"              26 minutes ago       Up 24 minutes (Paused)                    sharp_ellis
441984f4416d   centos:latest  "/bin/bash"              About an hour ago    Exited (0) About an hour ago              happy_greider
ce7a752a1115   centos:latest  "/bin/bash"              About an hour ago    Exited (0) 33 minutes ago                 flamboyant_easley
ffc02ff38c83   centos:latest  "echo 'hello! I want…"   About an hour ago    Exited (0) About an hour ago              amazing_aryabhata
448feb4421db   centos:latest  "echo 'hello! I am h…"   About an hour ago    Exited (0) About an hour ago              amazing_carver
8dd3224ae424   hello-world    "/hello"                 2 hours ago          Exited (0) 2 hours ago                    amazing_montalcini
[root@slave hfut]#
```

图 27 删除容器

4.5 导入和导出容器

某些时候，需要将容器从一个系统迁移到另外一个系统，此时可以使用 Doder 的导入和导出功能。

1. 导出容器

导出容器是指，导出一个已经创建的容器到一个文件，不管此时这个容器是否处于运行状态。可以使用 docker [container] export 命令，该命令格式为：

```
docker [container] export [-o]--output[=""]] CONTAINER
```

例子如下：

```
docker ps -a
docker export -o test_for_run.tar eb2504e5715f
```

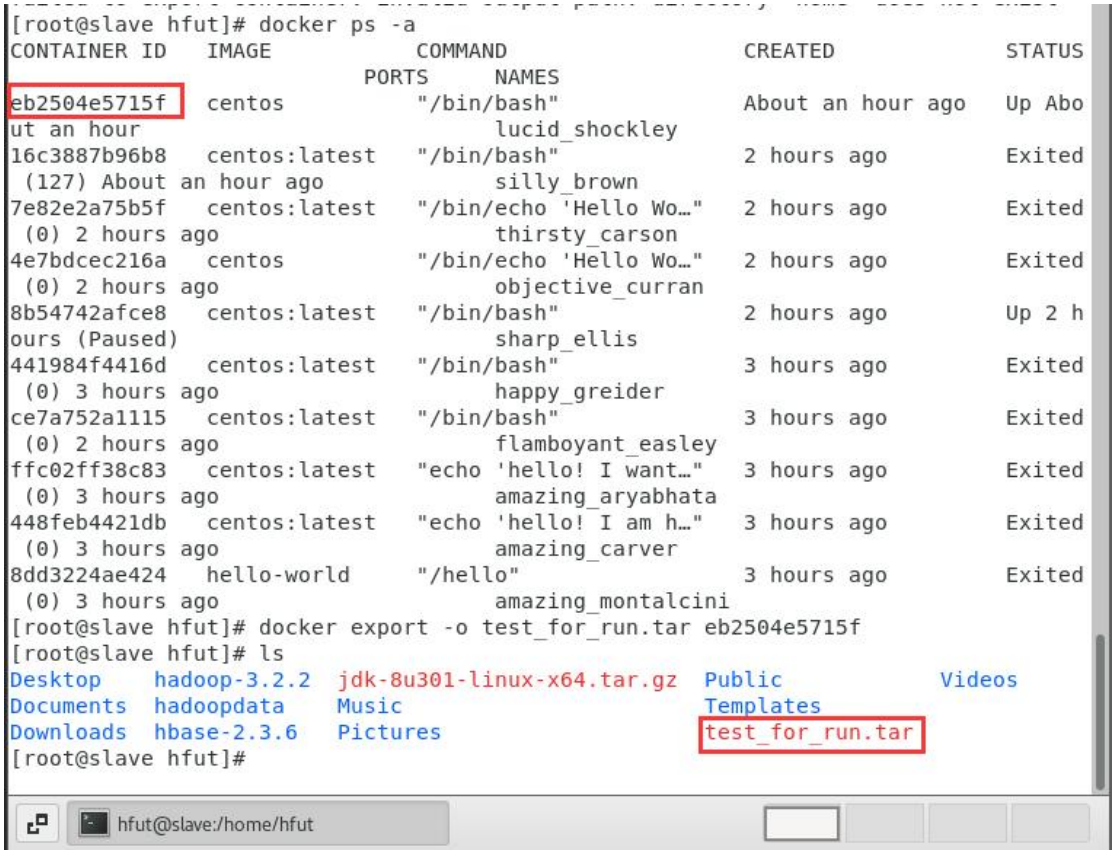


图 28 导出容器

之后，可将导出的 tar 文件传输到其他机器上，然后再通过导入命令导入到系统中，实现容器的迁移。

2. 导入容器

导出的文件又可以使用 docker [container] import 命令导入变成镜像，该命令格式为：

```
docker import [-c|--change[=[]]] [-m1--message [=MESSAGE]] file|URL| -
```

[REPOSITORY[:TAG]]

下例将刚才导出的 test_for_run.tar 文件导入到系统中：

```
docker import test_for_run.tar test/centos:ljj
```

```
[root@slave hfut]# docker import test_for_run.tar test/centos:ljj
sha256:3c195b2c0d4b76a435de499e70bc22e1eaf4709ffa3f4dfda41b1143064b73b1
[root@slave hfut]# docker images
REPOSITORY    TAG       IMAGE ID       CREATED          SIZE
test/centos   ljj       3c195b2c0d4b   49 seconds ago   231MB
test          0.1       36b957fc9f1c   3 hours ago      231MB
hello-world   latest    feb5d9fea6a5   9 months ago     13.3kB
centos        latest    5d0da3dc9764   9 months ago     231MB
[root@slave hfut]#
```

图 29：导入结果

5 Docker搭建大数据平台

5.1 Hadoop

我们在之前的大数据课上的实验中已经介绍并使用了hadoop，在此就不在赘述

5.1.1 节点搭建

（1）下载 Docker 镜像

```
sudo docker pull kiwenlau/hadoop:1.0
```

（2）下载 GitHub 仓库

```
git clone https://github.com/kiwenlau/hadoop-cluster-docker
```

（3）创建 Hadoop 网络

```
sudo docker network create --driver=bridge hadoop
```

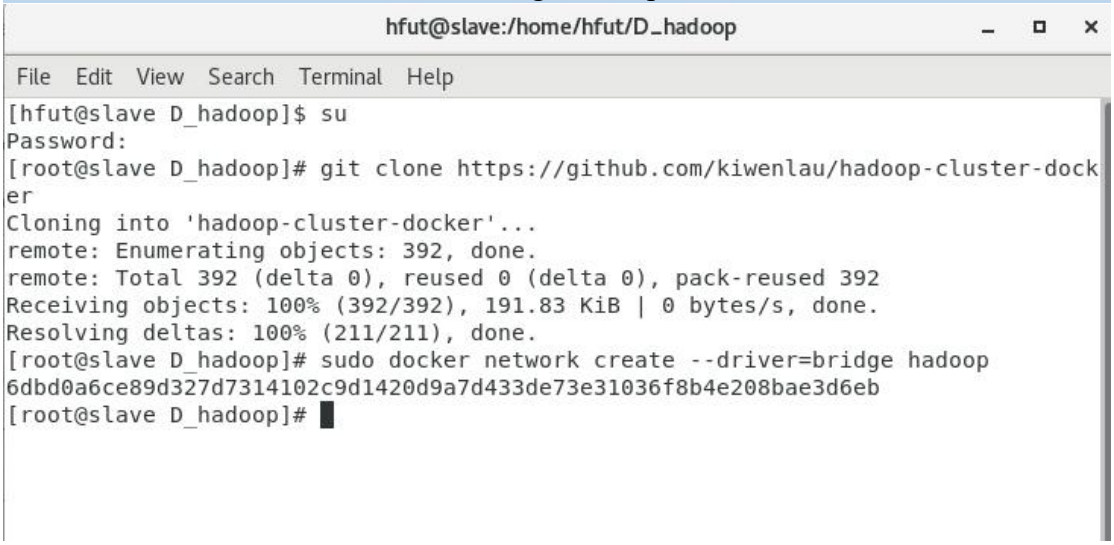


图 30 下载hadoop

（4）运行 Docker容器

```
cd hadoop-cluster-docker
./start-container.sh
```

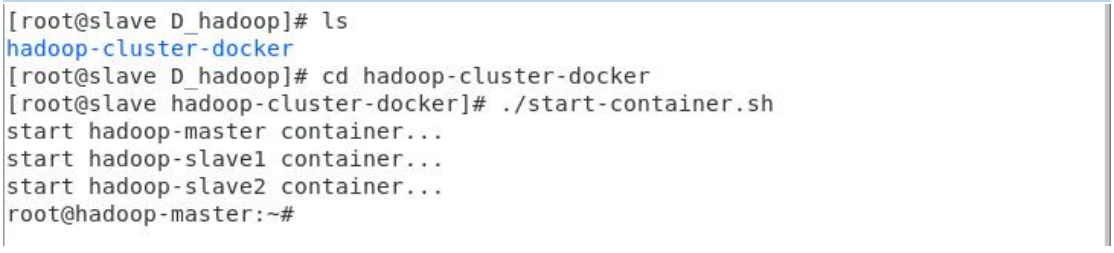


图 31 运行 Docker 容器

（5）启动 hadoop

./start-hadoop.sh

```
root@hadoop-master:~# ./start-hadoop.sh


Starting namenodes on [hadoop-master]
hadoop-master: Warning: Permanently added 'hadoop-master,172.18.0.2' (ECDSA) to the list of known hosts.
hadoop-master: starting namenode, logging to /usr/local/hadoop/logs/hadoop-root-namenode-hadoop-master.out
hadoop-slave1: Warning: Permanently added 'hadoop-slave1,172.18.0.3' (ECDSA) to the list of known hosts.
hadoop-slave2: Warning: Permanently added 'hadoop-slave2,172.18.0.4' (ECDSA) to the list of known hosts.
hadoop-slave2: starting datanode, logging to /usr/local/hadoop/logs/hadoop-root-datanode-hadoop-slave2.out
hadoop-slave1: starting datanode, logging to /usr/local/hadoop/logs/hadoop-root-datanode-hadoop-slave1.out
Starting secondary namenodes [0.0.0.0]
0.0.0.0: Warning: Permanently added '0.0.0.0' (ECDSA) to the list of known hosts.
0.0.0.0: starting secondarynamenode, logging to /usr/local/hadoop/logs/hadoop-root-secondarynamenode-hadoop-
master.out


starting yarn daemons
starting resourcemanager, logging to /usr/local/hadoop/logs/yarn--resourcemanager-hadoop-master.out
hadoop-slave2: Warning: Permanently added 'hadoop-slave2,172.18.0.4' (ECDSA) to the list of known hosts.
hadoop-slave1: Warning: Permanently added 'hadoop-slave1,172.18.0.3' (ECDSA) to the list of known hosts.
hadoop-slave1: starting nodemanager, logging to /usr/local/hadoop/logs/yarn-root-nodemanager-hadoop-slave1.o
ut
hadoop-slave2: starting nodemanager, logging to /usr/local/hadoop/logs/yarn-root-nodemanager-hadoop-slave2.o
ut


root@hadoop-master:~#
```

图 32 启动 Hadoop

（6）运行 wordcount

/run-wordcount.sh

```
root@hadoop-master: ~
File  Edit  View  Search  Terminal  Help
                Merged Map outputs=2
                GC time elapsed (ms)=160
                CPU time spent (ms)=1230
                Physical memory (bytes) snapshot=645767168
                Virtual memory (bytes) snapshot=2526720000
                Total committed heap usage (bytes)=296493056
        Shuffle Errors
                BAD_ID=0
                CONNECTION=0
                IO_ERROR=0
                WRONG_LENGTH=0
                WRONG_MAP=0
                WRONG_REDUCE=0
        File Input Format Counters
                Bytes Read=26
        File Output Format Counters
                Bytes Written=26

input file1.txt:
Hello Hadoop

input file2.txt:
Hello Docker

wordcount output:
Docker  1
Hadoop  1
Hello   2
root@hadoop-master:~#
```

图 33 运行 wordcount 结果

可以看见 Hadoop 集群已可正常使用。

5.1.2 N节点Hadoop集群搭建

（1）重新构建 Docker 镜像

```
./resize-cluster.sh N
```

可指定任N(N>1)，这里设定为5。

（2）启动Docker容器

```
./start-container.sh
```

```
[root@slave hadoop-cluster-docker]# ./start-container.sh
start hadoop-master container...
start hadoop-slave1 container...
start hadoop-slave2 container...
start hadoop-slave3 container...
start hadoop-slave4 container...
root@hadoop-master:~# ./start-hadoop.sh
```

图 34 5节点启动结果

（3）启动 hadoop，运行 wordcount

```
./start-hadoop.sh
./run-wordcount.sh
```

```
root@hadoop-master: ~
File  Edit  View  Search  Terminal  Help
                Merged Map outputs=2
                GC time elapsed (ms)=405
                CPU time spent (ms)=1470
                Physical memory (bytes) snapshot=648175616
                Virtual memory (bytes) snapshot=2529198080
                Total committed heap usage (bytes)=296493056
        Shuffle Errors
                BAD_ID=0
                CONNECTION=0
                IO_ERROR=0
                WRONG_LENGTH=0
                WRONG_MAP=0
                WRONG_REDUCE=0
        File Input Format Counters
                Bytes Read=26
        File Output Format Counters
                Bytes Written=26

input file1.txt:
Hello Hadoop

input file2.txt:
Hello Docker

wordcount output:
Docker  1
Hadoop  1
Hello   2
root@hadoop-master:~# a
```

图 35 5节点运行wordcount 结果

5.2 Spark

Spark，是一种"One Stack to rule them all"的大数据计算框架，期望使用一个技术堆栈就完美地解决大数据领域的各种计算任务。Apache官方,对Spark的定义就是：通用的大数据快速处理引擎。

Spark使用Spark RDD、Spark SQL、 Spark Streaming，MLlib，GraphX成功解决了大数据领域中，离线批处理、交互式查询、实时流计算、机器学习与图计算等最重要的任务和问题。

Spark除了一站式的特点之外，另外一个最重要的特点，就是基于内存进行计算，从而让它的速度可以达到MapReduce、Hive的数倍甚至数十倍！

（1）使用官方镜像

可使用 docker pull 指令直接获取镜像

```
docker pull sequenceiq/spark:1.6.0
```

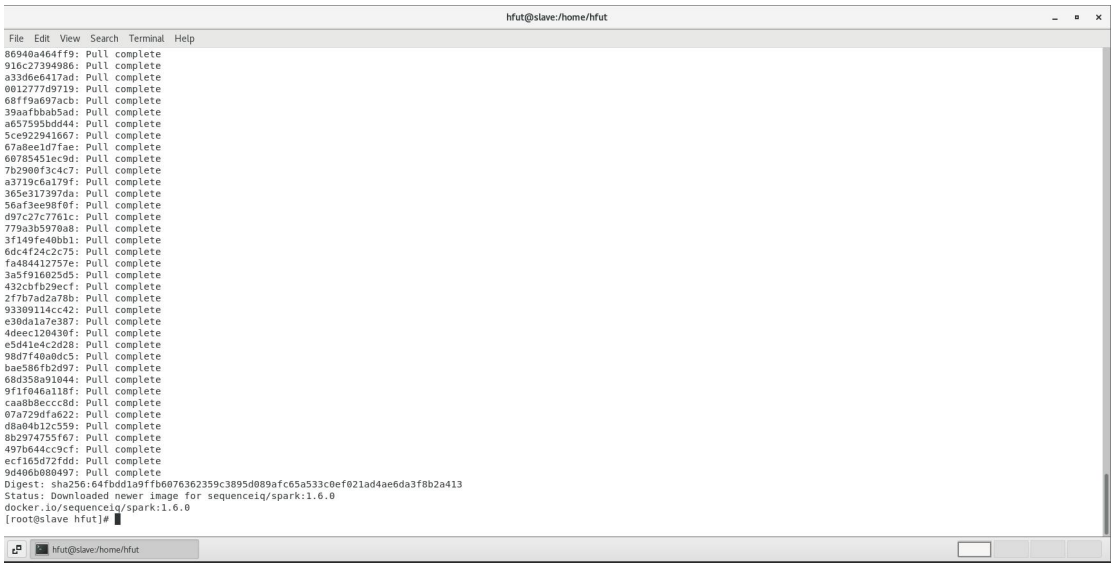


图 36 使用命令获取镜像

（2）在运行容器时，需要映射 YARN UI 需要的端口

```
docker run -it -p 8080:8088 -p 8042:8042 -h sandbox sequenceiq/spark:1.6.0 bash
```



图 37 映射结果

（3）启动后，可以使用 bash 命令行来查看 namenode 日志等信息

bash-4.1# cat /usr/local/hadoop/logs/hadoop-root-namenode-sandbox.out

```
bash-4.1# cat /usr/local/hadoop/logs/hadoop-root-namenode-sandbox.out
ulimit -a for user root
core file size          (blocks, -c) unlimited
data seg size           (kbytes, -d) unlimited
scheduling priority             (-e) 0
file size               (blocks, -f) unlimited
pending signals                 (-i) 7154
max locked memory       (kbytes, -l) 64
max memory size         (kbytes, -m) unlimited
open files                      (-n) 1048576
pipe size            (512 bytes, -p) 8
POSIX message queues     (bytes, -q) 819200
real-time priority              (-r) 0
stack size              (kbytes, -s) 8192
cpu time               (seconds, -t) unlimited
max user processes              (-u) unlimited
virtual memory          (kbytes, -v) unlimited
file locks                      (-x) unlimited
bash-4.1#
```

图 38 查看namenode 日志

（4）验证在 YARN 客户端模式中，SparkContext 运行在客户端进程中，主(master)应用仅处理来自 YARN 的资源管理请求：

bash-4.1# spark-shell --master yarn-client --driver-memory 1g --executor-memory 1g --executor-cores 1 scala> sc.parallelize(1 to 1000).count()

```
bash-4.1# spark-shell --master yarn-client --driver-memory 1g --executor-memory 1g

22/07/09 07:43:40 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
22/07/09 07:43:40 INFO spark.SecurityManager: Changing view acls to: root
22/07/09 07:43:40 INFO spark.SecurityManager: Changing modify acls to: root
22/07/09 07:43:40 INFO spark.SecurityManager: SecurityManager: authentication disabled; ui acls disabled; users with view permissions: Set(root); users with modify permissions: Set(root)
22/07/09 07:43:40 INFO spark.HttpServer: Starting HTTP Server
22/07/09 07:43:40 INFO server.Server: jetty-8.y.z-SNAPSHOT
22/07/09 07:43:40 INFO server.AbstractConnector: Started SocketConnector@0.0.0.0:43801
22/07/09 07:43:40 INFO util.Utils: Successfully started service 'HTTP class server' on port 43801.
Welcome to
      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /___/ .__/\_,_/_/ /_/\_\   version 1.6.0
      /_/

Using Scala version 2.10.5 (Java HotSpot(TM) 64-Bit Server VM, Java 1.7.0_51)
Type in expressions to have them evaluated.
Type :help for more information.
22/07/09 07:43:47 INFO spark.SparkContext: Running Spark version 1.6.0
22/07/09 07:43:47 INFO spark.SecurityManager: Changing view acls to: root
22/07/09 07:43:47 INFO spark.SecurityManager: Changing modify acls to: root
```

图 39 spark 启动

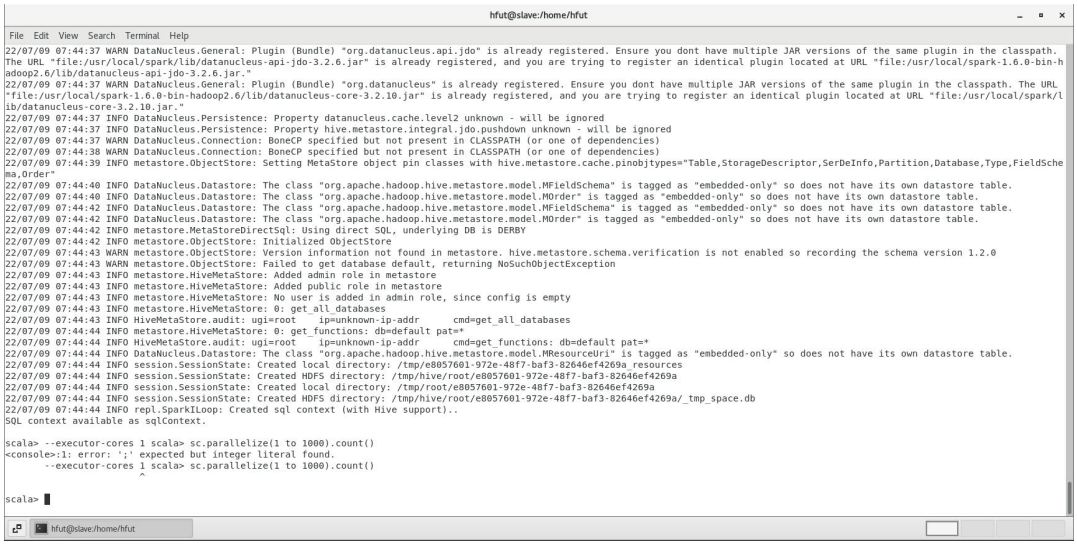


图 40 spark 运行结果

5.3 Storm

Apache Storm 是一个实时流计算框架，由 Twitter 在 2014 年正式开源，遵循 Eelipse Public License 1.0，基于 Clojure 等语言实现。

Storm 集群与 Hadoop 集群在工作方式上十分相似，唯一区别在于 Hadoop 上运行的是 MapReduce 任务,在 Storm 上运行的则是 topology。MapReduce 任务完成处理即会结束，而 topology 则永远在等待消息并处理（直到停止）。Storm 集群中有两种节点：主节点和工作节点，主节点运行一一个叫“Nimbus”的守护进程（daemon），与 Hadoop 的“任务跟踪器”(Jobtracker) 类似。Nimbus 负责向集群中分发代码，向各机器分配任务，以及监测故障。工作节点运行

“Supervisor”守护进程，负责监听 Nimbus 指派到机器的任务，根据指派信息来管理工作者进程（ worker process），每一个工作者进程执行一个 topology 的任务子集。

Nimbus 和 Supervisors 之间的所有协调调度通过 Zookeeper 集群来完成。另外，Nimbus 守护进程和 Supervisor 守护进程都是快速失败和无状态的，实现极高的稳定度。

（1）使用 Compose 搭建 Storm 集群

```
git clone https://github.com/denverdino/docker-storm.git
cd '/home/hfut/Strom/docker-storm/local'
```

```
[root@slave hfut]# git clone https://github.com/denverdino/docker-storm.git
bash: git: command not found...
[root@slave hfut]# yum install git
Loaded plugins: fastestmirror, langpacks
Loading mirror speeds from cached hostfile
 * base: mirrors.njupt.edu.cn
 * extras: mirrors.njupt.edu.cn
 * updates: mirrors.ustc.edu.cn
Resolving Dependencies
--> Running transaction check
---> Package git.x86_64 0:1.8.3.1-23.el7_8 will be installed
--> Processing Dependency: perl-Git = 1.8.3.1-23.el7_8 for package: git-1.8.3.1-23.el7_8.x86_64
--> Processing Dependency: perl(Term::ReadKey) for package: git-1.8.3.1-23.el7_8.x86_64
--> Processing Dependency: perl(Git) for package: git-1.8.3.1-23.el7_8.x86_64
--> Processing Dependency: perl(Error) for package: git-1.8.3.1-23.el7_8.x86_64
--> Running transaction check
---> Package perl-Error.noarch 1:0.17020-2.el7 will be installed
---> Package perl-Git.noarch 0:1.8.3.1-23.el7_8 will be installed
---> Package perl-TermReadKey.x86_64 0:2.30-20.el7 will be installed
--> Finished Dependency Resolution

Dependencies Resolved

================================================================================
 Package                Arch              Version                  Repository  Size
================================================================================
Installing:
 git                    x86_64            1.8.3.1-23.el7_8         base       4.4 M
Installing for dependencies:
 perl-Error             noarch            1:0.17020-2.el7          base        32 k
 perl-Git               noarch            1.8.3.1-23.el7_8         base        56 k
 perl-TermReadKey       x86_64            2.30-20.el7              base        31 k

Transaction Summary
================================================================================
Install  1 Package (+3 Dependent packages)

Total download size: 4.5 M
```

图 41 克隆下载Storm

（2）运行下列命令构建测试镜像

```
docker-compose build
```

（3）利用下列命令一键部署一个Storm 应用

```
docker-compose up -d
```

（4）部署完毕，检查 Storm 应用状态

```
docker-compose ps
```

（5）当 UI 容器启动后，可以访问当前机器的 8080 端口来打开操作界面：

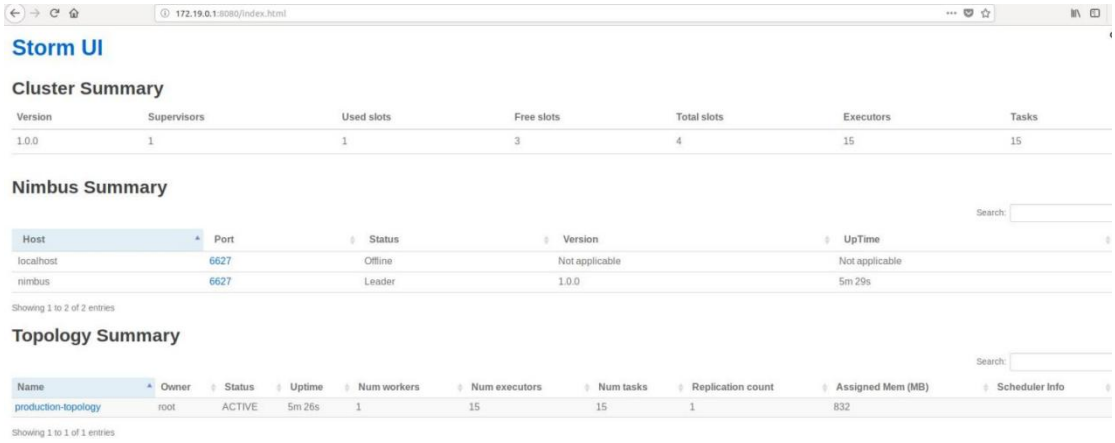


图 42 Strom 操作界面

6 写在最后

6.1 心得体会

这次的课程设计，我选择了自学一项大数据相关技术并撰写实验指导书；之所以会有这个想法主要是受到吴共庆老师撰写的实验指导书的启发：我想锻炼一下自己能力，而且撰写自己的实验指导书也许能为他人行方便；没想到，当初跟着吴老师的实验指导书顺风顺水的我，在自己钻研实验、撰写实验指导书时也会遇坑无数，总是刚爬出一个坑又掉入另一个坑，从早到晚；那是一天又一天，浏览器的搜索标签更是打开了一个又一个，密密麻麻到我以及不想寻找之前打开的页面在哪；我几度想要放弃，却总是会想到当初吴老师说他深夜凌晨还在为实验指导书爬坑，这让我又重新振作起来：老师可是在探索使用最新的工具来为我们撰写实验指导书，而我的是好几年前的大数据技术，网络上相关的资源比较丰富，我所遇到的错误，肯定已经有前辈走过了，只要肯花时间，我相信问题最终总是能够被解决的！终于，在我的努力下，皇天不负有心人。历时近5天，我成功做出了这一份关于Docker的实验指导书，虽然在排版、内容、严谨性、时效性、规范性等方面上都有许多瑕疵，不能称的上是一份标准的指导书，但这仍然是我呕心沥血之作，成就感满满！

通过这次实验指导书的撰写，大大提升了我对linux系统指令的操作熟练度、bug报错的迅速搜集能力、检查问题的耐心以及编写报告的能力，我感到受益匪浅！

6.2 参考材料

为了编写这次的实验指导书，我参考了很多网络上的资料，也多亏这些前辈们，我才没有在一个个大坑面前倒下，一步步走到了最后；以下是我罗列的一些参考材料，如果大家在做实验过程中遇到了无法解决的困难，也可以参考下列资料，寻求解决办法：

Docker概念：

(13条消息) docker是啥？是干什么的？_卡夫卡的熊kfk的博客-CSDN博客_docker是干什么的

Centos上的Docker安装：

在 CentOS | 上安装 Docker 引擎Docker 文档

Docker上手：

(13条消息) 【Docker技术入门与实践（第2版）】Docker入门 学习笔记

_He110_Wr0ld的博客-CSDN博客
(13条消息) Docker 学习笔记（CentOS 7.1）_weixin_34192816的博客-CSDN博客
(13条消息) Docker容器基本命令_我敲BUG的博客-CSDN博客_容器命令
(13条消息) Docker教程（超全总结）_生信技术的博客-CSDN博客_docker教程

Docker搭建大数据平台：

(13条消息) Spark工作原理及基础概念（超详细！）_<一蓑烟雨任平生>的博客-CSDN博客_spark原理
(13条消息) 基于docker的storm集群搭建_独醉F的博客-CSDN博客(13条消息) Docker 搭建大数据平台_蚂蚁搬运工的博客-CSDN博客_docker搭建大数据集群